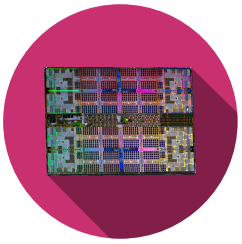

Single-Cycle RISC-V

Table des matières

1 Introduction **1**

2 Architecture **2**
2.1 Analyse 2

3 Control Unit **2**
3.1 Réalisation 3

4 Simulation **4**
4.1 Analyse 5

5 Déploiement (Facultatif) **6**

Acronymes **6**

1 Introduction

Ce laboratoire permet d'approcher l'architecture RISC-V à l'aide d'un processeur supportant un set d'instructions réduit, capable d'exécuter un petit programme assembleur en ne nécessitant qu'un seul coup de clock par instruction (single-cycle). Nommé `HEIRV32_SC`, ce dernier peut être simulé ou déployé directement sur une puce FPGA. La connexion avec le monde extérieur peut se faire à l'aide de boutons et de leds.

2 Architecture

Hormis l'incrémentation du program counter, le reste du circuit fonctionne de manière purement combinatoire. Le circuit est le suivant :

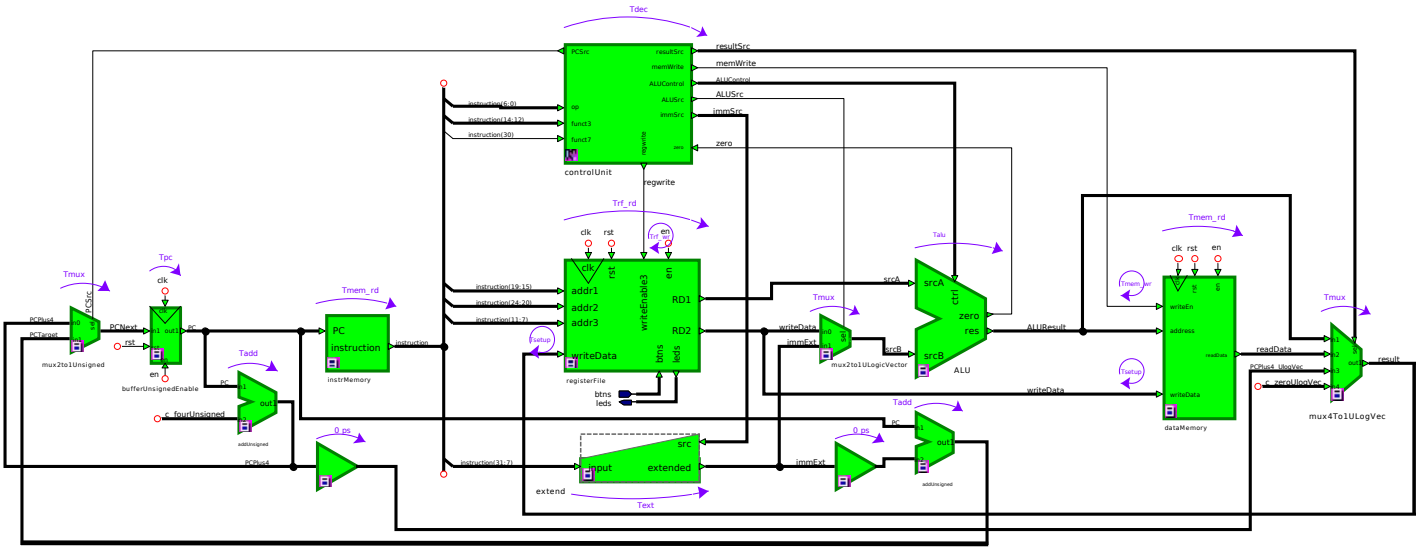

FIGURE 1 – Top level

2.1 Analyse

Déterminez les diverses fonctionnalités du circuit :

1. Quel est le rôle de chaque bloc ?
2. Identifiez les 5 étapes du pipeline RISC-V et quels blocs y sont liés.
3. A quoi servent chaque signaux ? Quelle est leur taille ?
4. Comment sont configurés les signaux du bloc **controlUnit** pour les instructions **or, addi, sw, beq, jal** ?

3 Control Unit

L'illustration 2 montre le bloc de contrôle qui reçoit tous les signaux d'entrée nécessaires pour générer les différents signaux de contrôle.

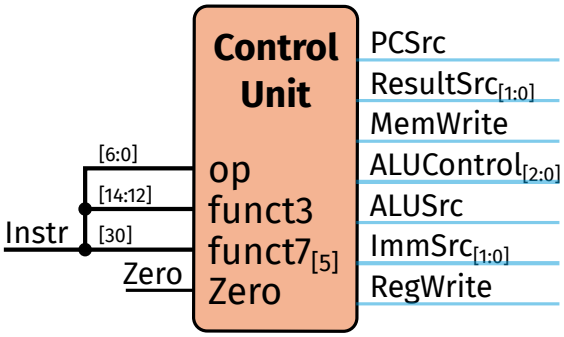

FIGURE 2 – Control Unit

Une des principale caractéristique différençiant un type de processeur d'un autre est la façon dont sont encodées les instructions, liée à la structure interne. Par exemple, l'opération d'addition d'un nombre avec un registre existe tout autant dans l'architecture RISC-V qu'en **x86** ou **ARM**, mais l'opcode généré est tout autre. Pour l'opération **addi x0 x0 1** en RISC-V (**add r0, r0, 1** en ARM), le résultat est :

- ARM : 0x 01 00 80 E2
- RISC-V : 0x 00 10 00 13

Le bloc **controlUnit** 2 est responsable de la configuration du circuit selon l'instruction donnée. Pour simplifier le circuit, **controlUnit** peut encore être divisé en deux sous-blocs **mainDecoder** et **ALUDecoder**, comme le montre l'illustration 3.

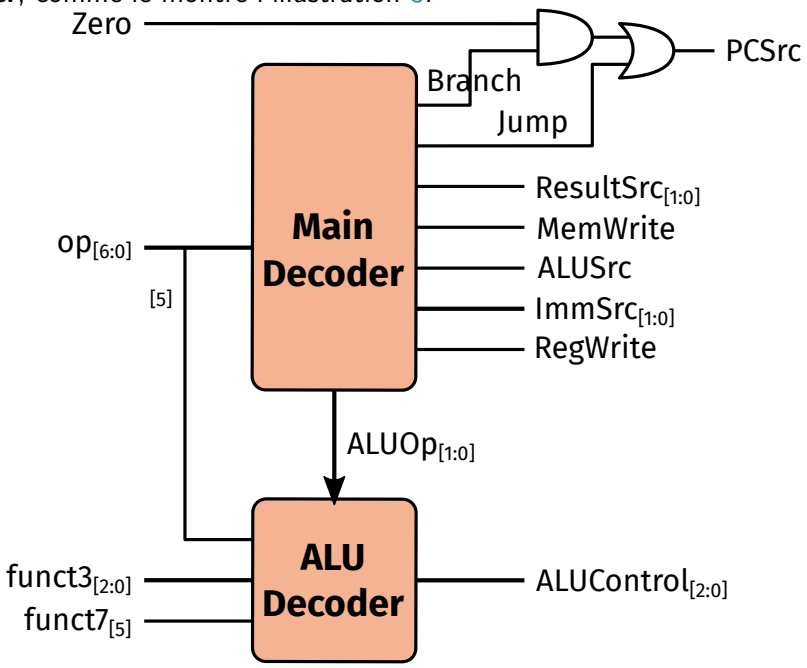

FIGURE 3 – Control Unit Detail

3.1 Réalisation

Complétez le bloc **controlUnit** afin de supporter les instructions suivantes :

- R-type : add, sub, slt, or, and
- I-type : addi, slti, ori, andi, lw
- S-type : sw
- B-type : beq
- J-type : jal

Pour cela, écrivez une table de vérité pour les deux sous-blocs, **mainDecoder** et **ALUDecoder**, pour toutes les instructions nécessaires.

Le **ALUDecoder** peut être réalisé à l'aide de multiplexeurs.

Le **mainDecoder** peut être écrit en VHDL. Pour cela, vous pouvez analyser l'exemple de code 4 ci-dessous et l'adapter en conséquence.

> Dans HDL Designer, lorsque vous sélectionnez le type de contenu d'un bloc, choisissez **VHDL File -> Architecture**, et contrôlez que le language est défini sur **VHDL 2008**. Sur la page suivante, **Architecture** correspond au nom de la vue (un bloc peut avoir différents contenus) et **Entity** au nom du bloc (mainDecoder par exemple).

```vhdl
-- Here our Entity is named "mainDecoder" and the Architecture "rtl"
-- Modify them according to your block
ARCHITECTURE rtl OF mainDecoder IS
BEGIN

process(op) -- we react on changes of op - purely combinatorial
begin
    case op is
        when "0000000" => -- when op corresponds to "0000000", do something (fake values)
            regwrite <= '0';
            immSrc <= "00";
            ALUSrc <= '0';
            memWrite <= '0';
            resultSrc <= "00";
            branch <= '0';
            ALUOp <= "00";
            jump <= '0';
        when others => -- in all cases not specified before, we let the synthesis pick 0 or 1 since we won't use it (do not forget a case !)
            regwrite <= '-';
            immSrc <= "--";
            ALUSrc <= '-';
            memWrite <= '-';
            resultSrc <= "--";
            branch <= '-';
            ALUOp <= "--";
            jump <= '-';
    end case;
end process;

END ARCHITECTURE rtl;
```

FIGURE 4 – Exemple de code MainDecoder

4 Simulation

Sous le dossier **Simulation** se trouvent les codes utilisés pour simuler et déployer le système, en trois formats :

- .s : code assembleur, en clair
- .bin : code compilé ; peut être analysé à l'aide d'outils comme Ghidra ou de désassembleurs en ligne
- .txt : version compilée compatible avec la mémoire de la FPGA pour le déploiement

Le code est compilé à l'aide du programme **HEIRV32-ASM**, disponible avec sa documentation sous le dossier du même nom.
Le testeur **heirv32_sc_tb** utilise le code **code_sim.bin** lors de la simulation :

```
# Base code :
  # riscvtest.s
  # Sarah.Harris@unlv.edu
  # David_Harris@hmc.edu
  # 27 Oct 2020

main:
  addi x2, x0, 5       # x2 = 5
  addi x3, x0, 12      # x3 = 12
  addi x7, x3, -9      # x7 = 12 - 9 = 3
  or   x4, x7, x2      # x4 = (3 or 5) = 7
  and  x5, x3, x4      # x5 = (12 AND 7) = 4
  add x5, x5, x4       # x5 = 4 + 7 = 11
  beq x5, x7, end      # shouldn't be taken
  slt x4, x3, x4       # x4 = (12 < 7) = 0
  beq x4, x0, around   # should be taken
  addi x5, x0,0        # shouldn't execute
around:
  slt x4, x7, x2       # x4 = (3 < 5) = 1
  add x7, x4, x5       # x7 = (1 + 11) = 12
  sub x7, x7, x2       # x7 = (12 - 5) = 7
  sw x7, 84(x3)        # [96] = 7
  lw x2, 96(x0)        # x2 = [96] = 7
  add x9, x2, x5       # x9 = (7 + 11) = 18
  jal x3, end          # jump to end, x3 = 0x44
  addi x2, x0,1        # shouldn't execute
end:
  add x2, x2, x9       # x2 = (7 + 18) = 25
  sw x2, 0x20(x3)      # [100] = 25
done:
  beq x2, x2, main     # infinite loop
```

4.1 Analyse

Analysez dans un premier temps le code **code_sim.s** afin d'en comprendre le contenu. Est-ce un bon candidat pour tester votre système ?
Déterminez le temps nécessaire pour effectuer la boucle du programme une fois, sachant que le système est cadencé à **66 MHz**.
Lancez la simulation et confirmez le fonctionnement du système.
Répondez aux questions suivantes :

1. Quelles sont les faiblesses de cette architecture single-cycle ?
2. Déterminez l'instruction la plus longue et retracez son chemin.
3. Quelle est la vitesse de clock maximale envisageable ?

> Les temps de chaque sous-parties du système sont donnés dans le top-level.

5 Déploiement (Facultatif)

Lors du déploiement sur la FPGA, le système sélectionne le code **code_sc.*** comme source. Analysez et comprenez le code en question. Que devrait-il arriver ?
La librairie **Board** contient le top level nécessaire au déploiement. Ouvrez-le et analysez le circuit : comment fonctionnent **en_step_n** et **en_full_n** ?
Le fichier **Board/concat/car-labs.lpf** contient la définition et emplacements des pins du système.
Le branchement de l'électronique se fait selon l'image suivante :

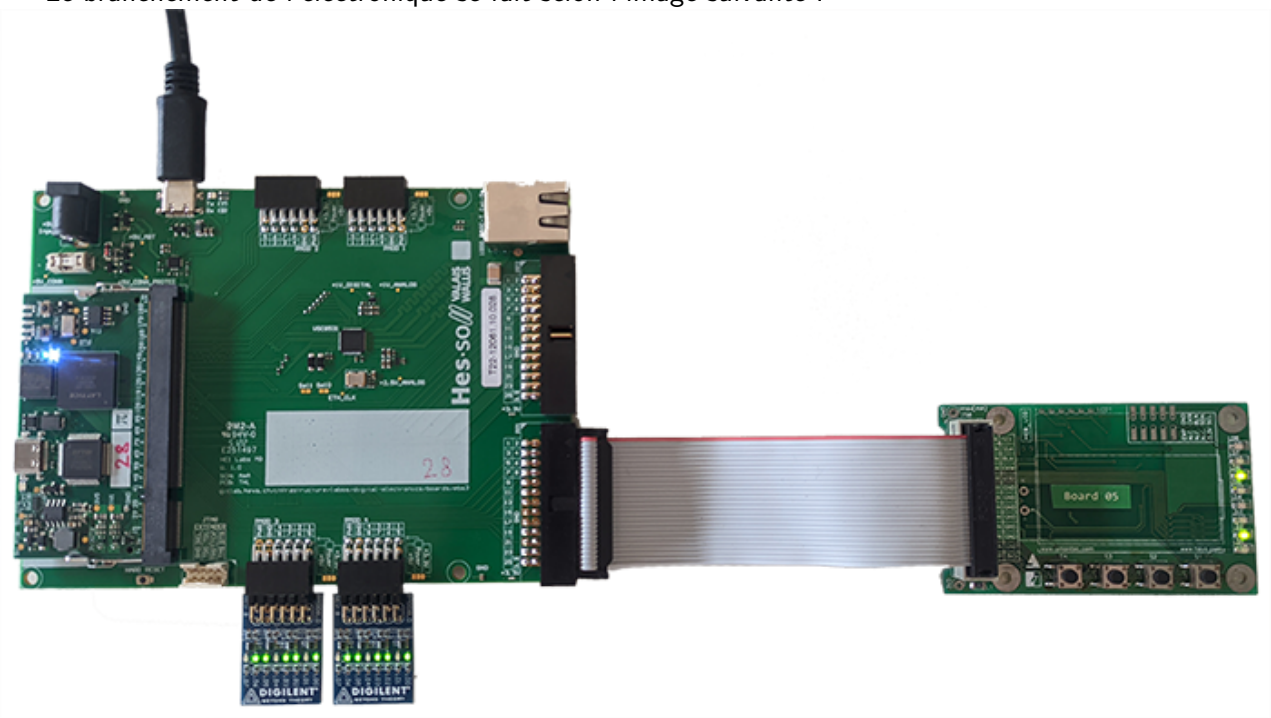

FIGURE 5 – Branchement du matériel

Déployez et testez le système sur la FPGA.

> Au démarrage du système, l'entrée **en** de HEIRV32_SC est à 0 !

Référez-vous au document **Board_LFE5u-25F.pdf** pour la synthèse et le flash par JTAG de la board.

Acronymes

FPGA **F**ield **P**rogrammable **G**ates **A**rray. 1, 4–6

RISC-V **R**educed **i**nstruction **s**et **c**omputer **V**. 1–3